PALADIN
BLOCKCHAIN SECURITY

Smart Contract Security Assessment

Final Report

For LayerZero (TempoOFTWrapper)

10 February 2026

paladinsec.co

info@paladinsec.co

Table of Contents

Table of Contents 2

Disclaimer 3

1 Overview 4

1.1 Summary 4

1.2 Contracts Assessed 4

1.3 Findings Summary 5

1.3.1 TempoOFTWrapper 6

1.3.2 ITempoOFTWrapper 6

2 Findings 7

2.1 TempoOFTWrapper 7

2.1.1 Privileged Functions 7

2.1.2 Issues & Recommendations 8

2.2 ITempoOFTWrapper 11

2.2.1 Issues & Recommendations 11

Disclaimer

Paladin Blockchain Security ("Paladin") has conducted an independent audit to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the codes that were provided for the scope of this audit. This audit report does not constitute agreement, acceptance or advocation for the Project that was audited, and users relying on this audit report should not consider this as having any merit for financial advice in any shape, form or nature. The contracts audited do not account for any economic developments that may be pursued by the Project in question, and that the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are completely free of exploits, bugs, vulnerabilities or deprecation of technologies. Further, this audit report shall not be disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Paladin.

All information provided in this report does not constitute financial or investment advice, nor should it be used to signal that any persons reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report. Information is provided 'as is', and Paladin is under no covenant to the completeness, accuracy or solidity of the contracts audited. In no event will Paladin or its partners, employees, agents or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this audit report.

Cryptocurrencies and any technologies by extension directly or indirectly related to cryptocurrencies are highly volatile and speculative by nature. All reasonable due diligence and safeguards may yet be insufficient, and users should exercise considerable caution when participating in any shape or form in this nascent industry.

The audit report has made all reasonable attempts to provide clear and articulate recommendations to the Project team with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts provided. It is the sole responsibility of the Project team to sufficiently test and perform checks, ensuring that the contracts are functioning as intended, specifically that the functions therein contained within said contracts have the desired intended effects, functionalities and outcomes of the Project team.

Paladin retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. Paladin is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. Paladin is furthermore allowed to claim bug bounties from third-parties while doing so.

1 Overview

This report has been prepared for LayerZero's contracts on the Tempo network. Paladin provides a user-centred examination of the smart contracts to look for vulnerabilities, logic errors or other issues from both an internal and external perspective.

1.1 Summary

Project Name	LayerZero
URL	https://layerzero.network/
Platform	Tempo
Language	Solidity
Preliminary Contracts	https://github.com/LayerZero-Labs/tempo-oft-wrapper/tree/5a37b6d7ffc9cde95319639a67fb64f1ad6d2155
Resolution #1	https://github.com/LayerZero-Labs/tempo-oft-wrapper/commit/69a0a4d223f6a658abbb8fea6ac2691bfe7e3fec

1.2 Contracts Assessed

- `TempoOFTWrapper.sol`
- `ITempoOFTWrapper.sol`

1.3 Findings Summary

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)
High	0	-	-	-
Medium	0	-	-	-
Low	0	-	-	-
Informational	3	1	-	2
Total	**3**	1	-	2

Classification of Issues

Severity	Description
High	Exploits, vulnerabilities or errors that will certainly or probabilistically lead towards loss of funds, control, or impairment of the contract and its functions. Issues under this classification are recommended to be fixed with utmost urgency.
Medium	Bugs or issues with that may be subject to exploit, though their impact is somewhat limited. Issues under this classification are recommended to be fixed as soon as possible.
Low	Effects are minimal in isolation and do not pose a significant danger to the project or its users. Issues under this classification are recommended to be fixed nonetheless.
Informational	Consistency, syntax or style best practices. Generally pose a negligible level of risk, if any.

1.3.1 TempoOFTWrapper

ID	Severity	Summary	Status
01	INFO	Missing mechanism to revoke cached maximum approvals	✓ RESOLVED
02	INFO	Missing explicit `_feeToken == ENDPOINT_DOLLAR` validation	ACKNOWLEDGED
03	INFO	Public vs external mismatch: interface inconsistency	ACKNOWLEDGED

1.3.2 ITempoOFTWrapper

No issues found.

2 Findings

2.1 TempoOFTWrapper

`TempoOFTWrapper` is a stateless wrapper contract deployed on Tempo that abstracts LayerZero's native fee payment for cross-chain OFT transfers. It takes a whitelisted stablecoin from the user, wraps it 1:1 into `LZEndpointDollar` (the LZ endpoint's ERC20 fee token) and executes the OFT send in a single transaction.

The contract caches maximum ERC20 approvals for gas efficiency and includes an owner-gated `recoverToken` function to withdraw any dust or mistakenly sent tokens.

2.1.1 Privileged Functions

- `recoverToken[onlyOwner]`
- `transferOwnership[onlyOwner]`
- `renounceOwnership[onlyOwner]`
- `acceptOwnership[pendingOwner]`

2.1.2 Issues & Recommendations

Issue #01	Missing mechanism to revoke cached maximum approvals
Severity	INFORMATIONAL
Description	`TempoOFTWrapper.sol#_maxApproveToken()` grants `type(uint256).max` allowance for a (`token`, `spender`) pair and then permanently caches it by setting `_isTokenMaxApproved[token][spender] = true`. Once set, it is never cleared and there is no owner function to revoke an approval. In `sendOFT()`, this results in permanent maximum approvals for `_feeToken -> ENDPOINT_DOLLAR` (trusted spender) and `ENDPOINT_DOLLAR -> _oft` (permissionless spender chosen by the caller). Over time, an unbounded number of `_oft` addresses accumulate permanent maximum `ENDPOINT_DOLLAR` spending rights on the wrapper. If `ENDPOINT_DOLLAR` ever ends up sitting in the wrapper (direct transfers or compose / refund flows - already accepted risks by the devs), it is exposed to all historically-approved `_oft` (or arbitrary contract address that imitate OFT interface but is malicious) spenders and the owner has no way to remove a specific spender's access (only withdrawing via `recoverToken` function). Theoretically, if each `transferFrom` operation reduces allowance when allowance was initially set to `type(uint256).max`, then after many transfers, the allowance can reach 0. The following check in `_maxApproveToken()`: `if (_isTokenMaxApproved[_token][_spender]) return;` will return early, and the wrapper will no longer be usable and will have to be redeployed. This is highly unlikely, since the fee tokens are expected to have 6 decimals and would require an unrealistic volume of fees to go through the contract for this to happen (technically possible).
Recommendation	Add an owner-gated function to revoke a specific approval and clear the cache flag.
Resolution	RESOLVED

Issue #02	Missing explicit `_feeToken == ENDPOINT_DOLLAR` validation
Severity	INFORMATIONAL
Description	The `sendOFT()` NatSpec explicitly states that the `_feeToken` parameter "reverts if it is `ENDPOINT_DOLLAR`". The scoping document also describes `_feeToken` as a whitelisted stablecoin, implying it is never `ENDPOINT_DOLLAR`. However, the implementation has no such check anywhere in `sendOFT()` function. What actually happens: if a user passes `_feeToken = ENDPOINT_DOLLAR`, the call will revert inside `LZEndpointDollar.sol#wrap()` with `NotWhitelisted(ENDPOINT_DOLLAR)`. The wrapper does not fail fast with its own validation, so users / integrators see a confusing revert reason from a different contract. Additionally, `_feeToken` is not validated as whitelisted before pulling tokens in the differen t-token path. If `_feeToken` is not whitelisted, the call still attempts two `safeTransferFrom` operations and approval work before reverting in `wrap()`. This is fine, but gas is wasted unnecessarily.
Recommendation	Add validation to check if `_feeToken` is `ENDPOINT_DOLLAR`. Optionally, also validate whitelisting upfront to fail fast on invalid fee tokens.
Resolution	ACKNOWLEDGED The NatSpec comments will be updated to explicitly state that the revert is handled by the `ENDPOINT_DOLLAR` contract rather than the wrapper itself.

Issue #03	Public vs external mismatch: interface inconsistency
Severity	INFORMATIONAL
Description	`ITempoOFTWrapper` interface declares these functions as external, but the `TempoOFTWrapper` implementation declares them public: `sendOFT()`, `recoverToken()`, `getEndpointDollar()` and `isTokenMaxApproved()`.
Recommendation	Change the implementation visibility to external where possible to match the interface.
Resolution	ACKNOWLEDGED The team has noted the recommendation but will maintain public visibility for its flexibility.

2.2 ITempoOFTWrapper

`ITempoOFTWrapper` is an interface for `TempoOFTWrapper` contract, defining the `sendOFT` function, `recoverToken` admin function, two view functions (`getEndpointDollar`, `getEndpointDollar`) and the custom errors and events used by the wrapper.

2.2.1 Issues & Recommendations

No issues found.

PALADIN
BLOCKCHAIN SECURITY